

[JOKER WEB SECURITY LAB]

مختبر الجوكر الأمني لتطبيقات الويب

(Joker Security Lab v2.0)

الدليل الأكاديمي والتطبيقي الشامل لتدريب الطلاب على أشهر ثغرات الويب
شرح نظري متعمق، سيناريوهات استغلال عملية، مقارنات الكود المصاب والأمن، ونظام الأعلام (CTF Flags)

المشروع مخصص للأغراض التعليمية والأمن الأخلاقي

إعداد وتطوير:

الجوكر الفلسطيني احمد سليم

بيئة عمل خفيفة مبنية بـ PHP و SQLite للتشغيل الفوري والنشر على الاستضافات

مقدمة عن مختبر الجوكر الأمني

وتطبيقية متكاملة تهدف إلى تزويد طلاب أمن المعلومات بتجربة عملية واقعية لاختبار اختراق تطبيقات الويب وفهم أسباب حدوث الثغرات الأمنية وكيفية ترقيعها برمجيا. يعد مختبر الجوكر الأمني منصة تدريبية

تعليمات في إعداد خوادم قواعد البيانات المنفصلة. يتيح ذلك تشغيل المختبر بنقرة واحدة محليا أو رفعه مباشرة على أي استضافة ويب مشتركة (Shared Hosting). تم بناء المنصة بلغة PHP وقاعدة بيانات SQLite 3 لتفادي أي

طريقة تشغيل وتثبيت المختبر:

1. التشغيل السريع محليا (Local Development):

افتح منفذ الأوامر داخل مجلد المشروع ونفذ الأمر:

php -S localhost:8000

ثم افتح المتصفح على الرابط: http://localhost:8000

2. النشر على استضافة ويب (Web Hosting / cPanel):

مجلد المشروع كاملا إلى مجلد public_html. سيعمل الموقع تلقائيا. تأكد من تفعيل صلاحيات الكتابة لمجلد uploads وملف قاعدة البيانات database.sqlite. ارفع

3. تنبيه أمني هام للمدرب:

تنفيذ أوامر حقيقية (RCE و Command Injection). إذا قمت بنشره على سيرفر عام متصل بالإنترنت، احرص على حمايته بكلمة مرور عبر Directory Privacy. المشروع يحتوي عمدا على ثغرات

1. حقن قواعد البيانات (SQL Injection - SQLi)

- SQL دون تعقيم أو استخدام استعلامات مجهزة (Prepared Statements). يتيح ذلك للمهاجم تعديل بنية الاستعلام وتنفيذ أوامر قاعدة بيانات غير مصرح بها.
- تحديث ثغرة SQLi عند دمج مدخلات المستخدم مباشرة داخل استعلام
- تجاوز شاشة الدخول (Auth Bypass): باستخدام البايلود الشهير: 'admin' -- في حقل اسم المستخدم، يتم إلغاء شرط فحص كلمة المرور وتسجيل الدخول كمدير.
- استخدام #&secret_code FROM products UNION SELECT 1, 2, 3, 4, x27; لاستخراج البيانات السرية والأعلام المخفية في جداول أخرى.
- استخراج البيانات عبر UNION:

الكود الآمن بعد الترقيع (Secure - High):	الكود المصاب (Vulnerable - Low):
<pre>\$stmt = \$db->prepare("SELECT * FROM users WHERE username = :u AND password = :p"); \$stmt->execute(["u" => \$user, "p" => md5(\$pass)]); \$row = \$stmt->fetch();</pre>	<pre>\$user = \$_POST["username"]; \$pass = \$_POST["password"]; \$sql = "SELECT * FROM users WHERE username = 'user' AND password = 'pass'"; \$db->query(\$sql);</pre>
:FLAG{SQLi_Auth_Bypass_Success_9281} العلم السري للتحدي (Flag):	

2. السكريبتات عبر المواقع (Cross-Site Scripting - XSS)

JavaScript خبيثة تنفذ في متصفحات الضحايا، مما يتيح سرقة ملفات الجلسة (Session Cookies) أو تشويه الصفحات أو توجيه المستخدمين لمواقع مشبوهة. تسمح للمهاجم بحقن أكواد

- <script>alert(1)</script> يحدث عند طباعة نص البحث مباشرة في الصفحة: (Reflected) XSS المنعكس
- حفظ التعليقات في قاعدة البيانات وعرضها للزوار دون ترميز، مما يجعل الهجوم دائما: (Stored) XSS المخزن
- innerHTML عبر جافاسكريبت وكتابتها عبر #name=... استخراج معاملات الرابط مثل: DOM المعتمد على XSS

الكود الآمن بعد الترقيع (Secure - High):	الكود المصاب (Vulnerable - Low):
--	----------------------------------

```
// ■■■■■ ■■■■■ ■■■ ■■■■■:
echo "<p>■■■■■ ■■■■■: " . $_GET["q"] . "</p>";
```

```
// ■■■■■■■■ htmlspecialchars ■■■■■■■■ ■■■■■■■■
■■■■■■■■ ■■■ ■■■■■■■■:
echo "<p>■■■■■■ ■■■■■■■■: " .
htmlspecialchars($_GET["q"], ENT_QUOTES, "UTF-8") .
"</p>";
```

العلم السري للتحدي (Flag): FLAG{XSS_Reflected_Payload_Found_7719}

3. تضمين الملفات واجتياز المسارات (LFI & Path Traversal)

(include, require) لقراءة ملفات حساسة من السيرفر عبر مسارات نسبية مثل ../../etc/passwd أو استخدام بروتوكولات PHP مثل filter.php.

استغلال دوال التضمين

- اجتياز المسارات (Path Traversal): طلب مسار نسبي: `file=secret_note.txt?` أو `win.ini/../../../../`
- مشغلات PHP المتقدمة: استخراج كود المصدر مشفرا: `php://filter/convert.base64-encode/resource=sqli.php`

الكود المصاب (Vulnerable - Low): <pre>\$file = \$_GET["file"]; include(\$file);</pre>	الكود الآمن بعد الترقيع (Secure - High): <pre>\$allowed = ["about.txt", "contact.txt"]; \$file = basename(\$_GET["file"]); if (in_array(\$file, \$allowed)) { include(\$file); } else { die("Access Denied"); }</pre>
العلم السري للتحدي (Flag): <code>FLAG{LFI_Local_File_Read_Exposed_8821}</code>	

4. رفع الملفات غير الآمن (Unrestricted File Upload)

(php.) دون التحقق الصارم من امتداد الملف أو محتواه الحقيقي، مما يمنح المهاجم شيل تحكم كامل (Web Shell) يؤدي إلى Remote Code Execution. السماح برفع ملفات برمجية تنفيذية

- رفع شيل PHP: إنشاء ملف shell.php يحتوي على دالة تنفيذ الأوامر ورفعها مباشرة كصورة بروفايل.

الكود المصاب :(Vulnerable - Low) <pre>\$dest = "uploads/" . \$_FILES["avatar"]["name"]; move_uploaded_file(\$_FILES["avatar"]["tmp_name"], \$dest);</pre>	الكود الآمن بعد الترفيع :(Secure - High) <pre>\$ext = strtolower(pathinfo(\$_FILES["avatar"]["name"], PATHINFO_EXTENSION)); if (in_array(\$ext, ["jpg","png"])) { \$safe_name = bin2hex(random_bytes(10)) . "." . \$ext; move_uploaded_file(\$_FILES["avatar"]["tmp_name"], "uploads/" . \$safe_name); }</pre>
العلم السري للتحدي :(Flag) FLAG{File_Upload_WebShell_RCE_5521}	

5. حقن أوامر نظام التشغيل (OS Command Injection)

استدعاء دوال النظام مثل shell_exec أو exec مع دمج مخدلات غير منقاة، مما يمكن المهاجم من دمج أو امر نظام إضافية باستخدام الرموز &&, amp;, |, .;

- حقن أمر عبر أداة Ping: إدخال: 127.0.0.1 & whoami أو 127.0.0.1 & dir لتففيذ أو امر السيرفر.

الكود المصاب (Vulnerable - Low): الكود الآمن بعد الترقيع (Secure - High):

```
$ip = $_POST["ip"];
$out = shell_exec("ping -c 1 " . $ip);
```

```
$ip = $_POST["ip"];
if (filter_var($ip, FILTER_VALIDATE_IP)) {
    $safe = escapeshellarg($ip);
    $out = shell_exec("ping -c 1 " . $safe);
}
```

:(Flag) العلم السري للتحدي FLAG{Command_Injection_Pwned_9912}

6. التحكم غير المباشر بالكائنات (IDOR / BOLA)

معرف الكائن (msg_id أو user_id) المرسل من المتصفح في جلب بيانات حساسة دون التأكد من أن صاحب الجلسة الحالية يملك صلاحية الاطلاع على هذا السجل. الاعتماد على

- قراءة رسائل الإدارة السرية: تغيير الرابط من msg_id=1? الخاص بالمستخدم إلى msg_id=2? لقراءة التقرير الأمني السري.

الكود المصاب (Vulnerable - Low):

```
$id = $_GET["msg_id"];
$msg = $db->query("SELECT * FROM messages WHERE
id = $id")->fetch();
```

الكود الآمن بعد الترقيع (Secure - High):

```
$id = (int)$_GET["msg_id"];
$uid = $_SESSION["user_id"];
$stmt = $db->prepare("SELECT * FROM messages WHERE
id = ? AND (receiver_id = ? OR sender_id = ?)");
$stmt->execute([$id, $uid, $uid]);
```

:(Flag) العلم السري للتحدي FLAG{IDOR_Unauthorized_Access_6619}

7. تزوير الطلبات عبر المواقع (CSRF)

متصفح الضحية المصادق عليه على إرسال طلب غير مرغوب فيه لتغيير بيانات حساسة (مثل البريد أو كلمة المرور) في غياب توكن الحماية Anti-CSRF Token. إجبار

- هجوم تغيير البريد الخفي: صفحة خارجية للمهاجم ترسل نموذجاً مخفياً يغير بريد المستخدم إلى hacker@evil.com.

الكود المصاب (Vulnerable - Low):

```
$new_email = $_POST["email"];
$db->query("UPDATE users SET email = '$new_email'
WHERE id = $uid");
```

الكود الآمن بعد الترقيع (Secure - High):

```
if (!hash_equals($_SESSION["csrf_token"],
$_POST["csrf_token"])) { die("Invalid Token"); }
// SameSite=Strict
```

:(Flag) العلم السري للتحدي FLAG{CSRF_Request_Forged_Successfully_4421}

8. كشف البيانات وقاعدة بيانات (SQLite Data Exposure)

في المسار العام للموقع دون منعه في إعدادات السيرفر، مما يتيح لأي زائر تحميل قاعدة البيانات كاملة والاطلاع على التجزئات المشفرة والبيانات السرية. ترك ملف قاعدة البيانات database.sqlite

- تحميل قاعدة البيانات المباشر: طلب الرابط مباشرة عبر المتصفح: http://example.com/database.sqlite

الكود المصاب (Vulnerable - Low):

```
# .htaccess
#
```

الكود الآمن بعد الترقيع (Secure - High):

```
# .htaccess:
<FilesMatch "\.(sqlite|db|env)$">
    Require all denied
</FilesMatch>
```

:(Flag) العلم السري للتحدي FLAG{Sensitive_Data_Exposed_Download_1192}

9. التوجيه المفتوح (Open Redirect)

توجيه الزائر إلى رابط خارجي يحدده المستخدم في المعامل دون التحقق من النطاق، مما يسهل عمليات التصيد الاحتيالي (Phishing) باستخدام نطاق الموقع الموثوق.

• رابط تصيد احتيالي: إرسال رابط يبدأ باسم الموقع الموثوق: `redirect.php?target=https://phishing-site.com`

الكود المصاب (Vulnerable - Low):	الكود الآمن بعد الترقيع (Secure - High):
<pre>\$url = \$_GET["target"]; header("Location: " . \$url);</pre>	<pre>\$url = \$_GET["target"]; if (strpos(\$url, "/") === 0 && strpos(\$url, "//") !== 0) { header("Location: " . \$url); }</pre>
العلم السري للتحدي (Flag): <code>FLAG{Open_Redirect_Exploited_Safe_3301}</code>	

10. تزوير الطلب من جانب الخادم (SSRF)

استغلال ميزات جلب الروابط من السيرفر لطلب عناوين شبكة داخلية خاصة (Localhost أو Private IPs) واختراق خدمات داخلية أو فحص المنافذ.

• فحص الخدمات الداخلية: إدخال `http://127.0.0.1` أو `http://localhost` لجعل السيرفر يطلب نفسه داخليا.

الكود المصاب (Vulnerable - Low):	الكود الآمن بعد الترقيع (Secure - High):
<pre>\$url = \$_POST["url"]; \$content = file_get_contents(\$url);</pre>	<pre>\$ip = gethostbyname(parse_url(\$url, PHP_URL_HOST)); if (!filter_var(\$ip, FILTER_VALIDATE_IP, FILTER_FLAG_NO_PRIV_RANGE FILTER_FLAG_NO_RES_RANGE)) { die("Private IPs Blocked!"); }</pre>
العلم السري للتحدي (Flag): <code>FLAG{SSRF_Internal_Server_Request_7710}</code>	

11. التخمين وغياب حماية المحاولات (Brute Force)

غياب آليات الحد من تكرار المحاولات (Rate Limiting) ورموز التحقق البشري (CAPTCHA)، مما يتيح تجربة قواميس كلمات مرور شائعة لكسر حساب المدير.

• تخمين كلمة مرور الحساب: استخدام كلمة مرور ضعيفة `admin123` ضد حساب `admin` عبر Burp Suite أو قاموس تجريبي.

الكود المصاب (Vulnerable - Low):	الكود الآمن بعد الترقيع (Secure - High):
<pre>if (\$user === "admin" && \$pass === \$db_pass) { login(); }</pre>	<pre>if (\$_SESSION["failed_attempts"] >= 3) { \$_SESSION["lockout"] = time() + 60; die("Account Locked for 60 seconds"); }</pre>
العلم السري للتحدي (Flag): <code>FLAG{Brute_Force_Password_Cracked_2281}</code>	

12. مقارنات PHP الضعيفة والأنواع (Type Juggling)

خلل في منطق المقارنة ناجم عن استخدام المقارنة الضعيفة (`==`) أو دوال مثل `strcmp` مع مصفوفات أو مقارنة سلاسل تبدأ بـ `0e` (Magic Hashes).

• تجاوز `strcmp` بالمصفوفة: إرسال `api_key[]=bypass` في POST لتمرير مصفوفة تجعل الدالة تتجاوز الفحص.

• الهاش السحري Magic Hash: إرسال نص `240610708` الذي ينتج عنه MD5 يبدأ بـ `0e` ويساوي صفرا في المقارنة الضعيفة.

الكود المصاب (Vulnerable - Low):	الكود الآمن بعد الترقيع (Secure - High):
----------------------------------	--

```
if (strcmp($secret, $_POST["api_key"]) == 0) {  
    grant(); }  
}
```

```
if (is_string($_POST["api_key"]) &&  
    hash_equals($secret, $_POST["api_key"])) { grant();  
}
```

علم السري للتحدي (Flag): `FLAG{PHP_Type_Juggling_Bypassed_6672}`

13. التلاعب بالصلاحيات والكوكي (Broken Access Control)

تخزين رتبة المستخدم أو صلاحياته في ملف تعريف ارتباط (Cookie) لدى المتصفح دون تشفير، مما يتيح للطالب تعديل قيمته من student إلى admin. • تصعيد الصلاحيات (Privilege Escalation): تعديل الكوكي user_role من student إلى admin عبر F12 DevTools لدخول لوحة التحكم.

الكود المصاب (Vulnerable - Low):

```
$role = $_COOKIE["user_role"];  
if ($role === "admin") { show_admin_panel(); }
```

الكود الآمن بعد الترقيع (Secure - High):

```
// Session Session Session Session  
Session:  
if ($_SESSION["user"]["role"] !== "admin") {  
    die("Access Denied"); }  
}
```

علم السري للتحدي (Flag): `FLAG{Privilege_Escalation_Admin_Role_5502}`

جدول الأعلام والحلول المرجعية السريعة (CTF Cheatsheet)

هذا الجدول مرجع خاص للمدرب لمتابعة حلول الطلاب والتأكد من الأعلام المحصلة لكل تحد:

التحدي	طريقة الحل / البايكود المقترح	CTF Flag
SQLi Auth	اسم المستخدم: 'admin' --	FLAG{SQLi_Auth_Bypass_Success_9281}
SQLi UNION	' UNION SELECT 1,'x','cat',0,secret_code FROM products --	FLAG{SQLi_Union_Extract_Secret_4812}
XSS Reflected	<script>alert(1)</script>	FLAG{XSS_Reflected_Payload_Found_7719}
XSS Stored	نشر كود جافاسكريبت في التعليقات	FLAG{XSS_Stored_Script_Triggered_3381}
XSS DOM	تحديث الرابط بـ #	FLAG{XSS_DOM_Execution_Caught_1928}
LFI	طلب الملف: file=secret_note.txt?	FLAG{LFI_Local_File_Read_Exposed_8821}
File Upload	رفع ملف shell.php واستدعاؤه في المتصفح	FLAG{File_Upload_WebShell_RCE_5521}
Command Inj	إرسال: amp;& whoami& 127.0.0.1	FLAG{Command_Injection_Pwned_9912}
IDOR	تعديل المعامل في الرابط إلى msg_id=2?	FLAG{IDOR_Unauthorized_Access_6619}
CSRF	إرسال طلب POST لتعديل البريد إلى hacker@evil.com	FLAG{CSRF_Request_Forged_Successfully_4421}
Sensitive Data	تحميل ملف database.sqlite مباشرة من الرابط	FLAG{Sensitive_Data_Exposed_Download_1192}
Open Redirect	طلب: target=https://google.com?	FLAG{Open_Redirect_Exploited_Safe_3301}
SSRF	طلب السيرفر الداخلي: http://127.0.0.1	FLAG{SSRF_Internal_Server_Request_7710}
Brute Force	تخمين كلمة مرور admin: admin123	FLAG{Brute_Force_Password_Cracked_2281}
Type Juggling	إرسال مصفوفة api_key[]=bypass أو هاش 240610708	FLAG{PHP_Type_Juggling_Bypassed_6672}
Access Control	تعديل الكوكي user_role إلى admin	FLAG{Privilege_Escalation_Admin_Role_5502}

تم بحمد الله وتوفيقه

نتمنى أن يكون هذا المختبر دليلاً نافعاً وممتعاً في تعليم واحتراف أمن تطبيقات الويب.

إعداد: الجوكر الفلسطيني احمد سليم